



BMCS2013 Data Engineering
CLO2 ASSIGNMENT 202409

Assignment Title : **Sinar Harian**

Team Reference : **G4-2**

Programme / Group : **RDS2 G4**

Student Name	Student ID	Contribution	Signature
Kenny Loh Kit Yi	24WMR08868	25%	<i>Kenny</i>
Kelly Tan Jie Li	24WMR08867	25%	<i>kelly</i>
Chin Yin Ern	24WMR08855	25%	<i>ern</i>
Chan Jin Wei	24WMR08851	25%	<i>wei</i>
TOTAL (must sum to 100%)		100%	

Kenny Loh Kit Yi	Kelly Tan Jie Li	Chin Yin Ern	Chan Jin Wei
			

1. Data Collection and Preparation

1.1. Data Source(s)

- Data source: Sinar Harian
- URL: <https://www.sinarharian.com.my>
- Brief description (include a screenshot):

Dapatkan Newsletter Percuma

Sinar Harian

Log Masuk Daftar

Ahad | 22 Disember 2024

PRU Sinar Daily SinarPlus Sinar Bestari

E-Paper Langgan

Utama Terkini Berita Suara Sinar Sinar Islam Podcast Khas Video Berita Premium Untuk Anda

- Iklan -

MongoDB 8.0 on Atlas.
More of what developers love at lower cost and less friction.

MongoDB Atlas

Sebuah helikopter terhempas di sebuah hospital di wilayah Mugla, barat daya Turkiye, mengorbankan empat orang.

GLOBAL 22 Disember 2024 06:26pm

Helikopter terhempas di hospital di Turkiye, empat maut

MUGLA - Sebuah helikopter terhempas di sebuah hospital di wilayah Mugla, barat daya Turkiye, mengorbankan empat orang, menurut Gabenor wilayah itu, İdris Akbiyik. Dua juruterbang....

22 Disember 2024 04:35pm

Thailand hormati keputusan Malaysia tutup pangkalan haram di sempadan

22 Disember 2024 03:31pm

Pelbagai perancangan strategik lonjakan ekonomi sepanjang 2024 - PM

22 Disember 2024 02:18pm

Kerajaan salur bantuan RM3.17 juta kepada 90 rumah ibadat bukan Islam

22 Disember 2024 01:03pm

Pelajar Tingkatan Enam terima BAP mulai Februari 2025

22 Disember 2024 07:08pm

22 Disember 2024 04:42pm

AMD Ryzen 7 7840U

SAVE 52%

Lenovo

Copilot+PC

Terkini Popular

Tulisan Jawi: Wajar kaji semula laksana secara pilihan

11 Ogos 1901 04:03pm

Gagal tolak tawaran lumayan punca rasuah berleluasa

03 Mei 1910 10:00am

Gangguan seksual: Zambry arah UM laksana siasatan segera

2024-12-22 01:00:00

'Mummy sudah cukup untuk Rayyan & Arjuna.' Anak enggan Rita Rudal...

22 Disember 2024 05:56pm

- Python libraries/API used:

- request
- Bs4
- Pyspark
- Re
- Concurrent
- Neo4j

- MediaWiki api
- Pandas
- Malaya api
- Malaya
- Subprocess
- Os
- Csv
- Matplotlib
- Kafka
- Time
- Stream_Process
- Threading

1.2. List of Python classes and Jupyter notebook(s) for data collection, cleaning and preprocessing:

Name of Python class(es) / notebooks	Author
DataSource.ipynb	Kenny Loh Kit Yi
web_scraper.py	Kenny Loh Kit Yi
data_crawler.py	Kenny Loh Kit Yi
DataCleaning.ipynb	Chan Jin Wei
data_cleaner.py	Chan Jin Wei

1.3. Code for Python Classes

webscraper.py

```
import requests
from bs4 import BeautifulSoup
from data_crawler import DataCrawler

class WebScraper:
    @staticmethod
    def extract_article_content(content_url):
        response = requests.get(content_url)
        soup = BeautifulSoup(response.text, 'html.parser')
        article_content = soup.find('article')

        for div in article_content.find_all('div'):
            div.decompose()

        p_tags = article_content.find_all('p', style=False)
        content = ' '.join([p.get_text(strip=True) for p in p_tags])
        return content

    @staticmethod
    def extract_all_articles(web_url, category_links, max_articles):
        article_contents = []
        total_articles = 0

        for category_link in category_links:
            if total_articles >= max_articles:
                break

            print(f"Processing category: {category_link}")
            article_links = DataCrawler.extract_article_links(web_url, category_link)
            for article_link in article_links:
                if total_articles >= max_articles:
                    break

                content = WebScraper.extract_article_content(article_link)
                if content:
                    article_contents.append(content)
                    total_articles += 1

        return article_contents
```

data_crawler.py

```
import requests
from bs4 import BeautifulSoup

class DataCrawler:
    @staticmethod
    def extract_category_links(web_url):
        response = requests.get(web_url)
        category_links = []

        soup = BeautifulSoup(response.text, 'html.parser')
        menu_buttons = soup.find('div', class_="menuButtons")
        navbar = menu_buttons.find('div', id="navbarContent")
        berita_section = navbar.find('a', string="Berita")
        subcategories = berita_section.find_next('ul')
        subcategory_items = subcategories.find_all('a')

        for subcategory in subcategory_items:
            subcategory_name = subcategory.get_text(strip=True).lower().replace(" ", "-")
            category_links.append(f"{web_url}/{subcategory_name}")

        return category_links

    @staticmethod
    def extract_article_links(web_url, article_url):
        response = requests.get(article_url)
        article_links = set()

        soup = BeautifulSoup(response.text, 'html.parser')
        article_links = {
            link['href'] for link in soup.find_all('a', href=True)
            if link['href'].startswith(f"{web_url}/article/")
        }

        return list(article_links)
```

data_cleaner.py

```
from pyspark.sql.functions import *

class DataCleaner:

    @staticmethod
    def csvToDataFrame(csv_file_path, spark):

        df = spark.read.csv(csv_file_path, header=True, inferSchema=True)

        return df

    @staticmethod
    def remove_numbers(df, column):

        return df.withColumn(column, regexp_replace(col(column), r'\d', " "))

    @staticmethod
    def convert_to_lowercase(df, column):

        return df.withColumn(column, lower(col(column)))

    @staticmethod
    def remove_standalone_characters(df, column):
        # Remove standalone characters (single characters surrounded by whitespace or at start/end of string)
        return df.withColumn(
            column,
            # Replace standalone single characters (surrounded by spaces, punctuation, or start/end)
            regexp_replace(
                col(column),
                r'(?<\w)(\b\w\b)(?!<\w)', # Matches standalone characters
                " "
            )
        )

    @staticmethod
    def remove_words_in_parentheses(df, column):
        # Remove words or any content in between ( )
        return df.withColumn(
            column,
            regexp_replace(col(column), r'\([^)]*\)', " ")
        )

    @staticmethod
    def remove_delimiter(df, column, delimiters):
        # Print debug message
        print(f"Removing delimiters: {delimiters} from column: {column}")

        # Build regex pattern and apply regexp_replace
        pattern = f"[{delimiters}]"
        return df.withColumn(column, regexp_replace(col(column), pattern, " "))

    @staticmethod
    def smart_remove_delimiter(df, column, delimiters):
        # Combine patterns into a single regex to handle all cases:
        # 1. Delimiters surrounded by non-word characters or spaces.
        # 2. Delimiters at the start of a string.
        # 3. Delimiters at the end of a string.
        pattern = f"^(?<[delimiters])|([delimiters])$|(?<\w)[delimiters]+(?!<\w)"

        # Replace matching delimiters with a space
        df = df.withColumn(column, regexp_replace(col(column), pattern, " "))

        return df
```

```

@staticmethod
def remove_hyphen_from_prefix(df, column, prefix="ke-"):

    # Regex pattern to match the exact prefix (e.g., "ke-")
    pattern = f"\\b{prefix}"

    # Replace the prefix with the version without the hyphen
    replacement = prefix.replace("-", "")

    df = df.withColumn(column, regexp_replace(col(column), pattern, replacement))

    return df

@staticmethod
def normalize_spaces(df, column):
    return df.withColumn(
        column,
        trim(regexp_replace(col(column), r'\s+', ' '))
    )

@staticmethod
def remove_duplicates(df, column):
    if column:
        return df.dropDuplicates([column])
    else:
        return df.dropDuplicates()

```

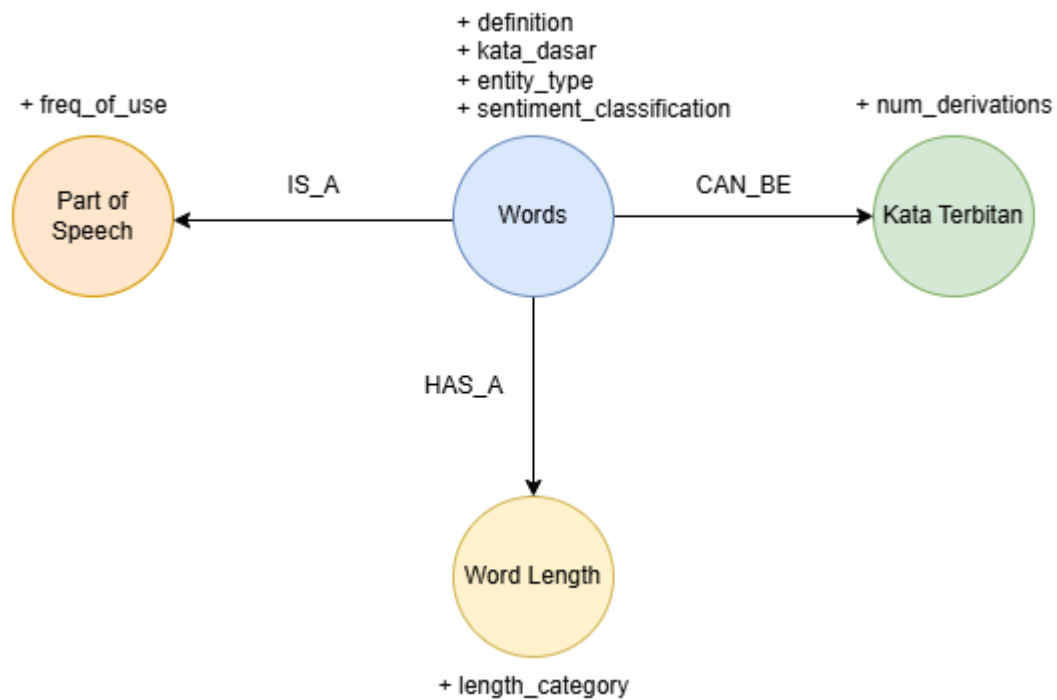
1.4. Data Quantity

```
df_split.count()
```

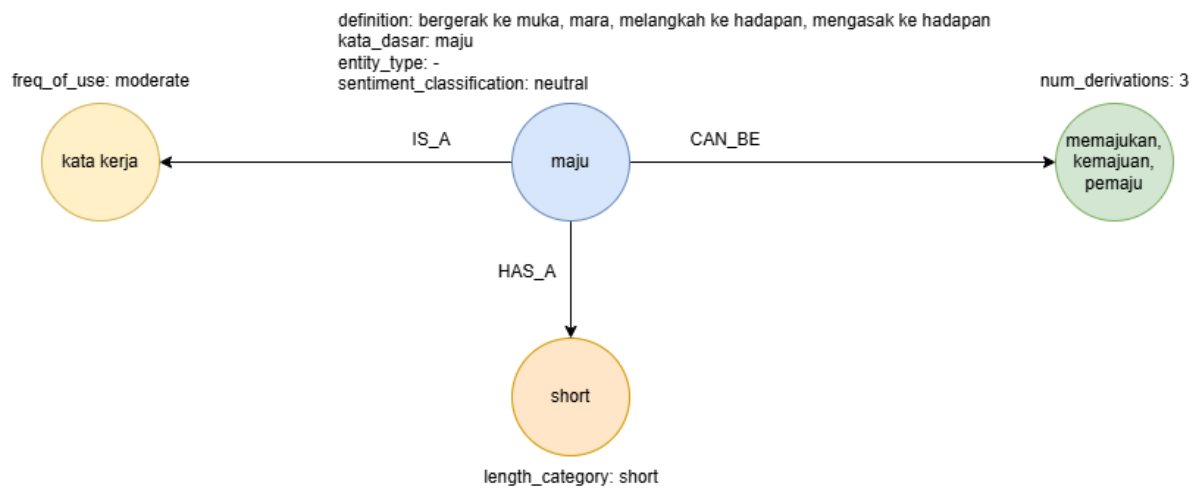
```
10382
```


2. Lexicon Creation

2.1. Data model design



2.2. Example of an entry in the data store



2.3. List of Python classes and Jupyter notebook(s) for lexicon:

Name of Python class(es) / notebooks	Author
FetchDefinition.ipynb	Chin Yin Ern
clean_dictionary.py	Chin Yin Ern
Neo4j.ipynb	Chin Yin Ern
neo_4j.py	Chin Yin Ern

2.4. Code for Python Classes

clean_dictionary.py

```
import requests
from bs4 import BeautifulSoup
import re
from pyspark.sql.functions import udf, col
from pyspark.sql.types import StringType
from concurrent.futures import ThreadPoolExecutor
from classes.dataframe_saver import DataFrameSaver

class CleanDefinition:
    @staticmethod
    def clean_definition(text):
        cleaned_text = re.sub(r'[\u0600-\u06FF\u0750-\u077F]', '', text)
        cleaned_text = re.sub(r'^\w\s\[.:\-~]', '', cleaned_text)
        return cleaned_text.strip()

    @staticmethod
    def fetch_definition(keyword):
        url = f"https://prpm.dbp.gov.my/Caril?keyword={keyword}"
        try:
            response = requests.get(url, timeout=5)
            response.raise_for_status()

            soup = BeautifulSoup(response.content, 'html.parser')
            s = soup.find('div', class_='tab-content')

            if s:
                raw_text = s.get_text(strip=True)
                return CleanDefinition.clean_definition(raw_text)
            else:
                return "-"
        except requests.RequestException:
            return "Error fetching content."

    @staticmethod
    def fetch_definitions_parallel(words, max_workers=10):
        with ThreadPoolExecutor(max_workers=max_workers) as executor:
            results = list(executor.map(CleanDefinition.fetch_definition, words))
        return results

    @staticmethod
    def fetch_and_save_definitions(spark, input_file, output_file_local, output_file_hdfs):
        input_df = spark.read.csv(input_file, header=True, inferSchema=True)

        unique_words = input_df.select("words").distinct().rdd.flatMap(lambda x: x).collect()

        definitions = CleanDefinition.fetch_definitions_parallel(unique_words, max_workers=10)

        word_definition_map = dict(zip(unique_words, definitions))

        def lookup_definition(word):
            return word_definition_map.get(word, "Definition not found")

        lookup_definition_udf = udf(lookup_definition, StringType())

        output_df = input_df.withColumn("definition", lookup_definition_udf(col("words")))

        output_dir1 = "/home/student/de-assgt/content/"
        file_name = "dictionary.csv"
        DataFrameSaver.save_to_csv(output_df, output_dir1, file_name)

        # Write the output DataFrame back to HDFS
        output_df.coalesce(1).write.csv(output_file_hdfs, header=True, mode="overwrite")
```

neo4j.py

```
from pyspark.sql import SparkSession
from neo4j import GraphDatabase

class Neo4j:
    driver = None

    @staticmethod
    def create_connection(uri, user, password):
        global driver
        AUTH = (user, password)
        Neo4j.driver = GraphDatabase.driver(uri, auth=AUTH)
        Neo4j.driver.verify_connectivity()
        return Neo4j.driver

    @staticmethod
    def close_connection():
        if Neo4j.driver is not None:
            Neo4j.driver.close()

    @staticmethod
    def write_to_neo4j(df, epoch_id, node, relationship=None, properties=None):
        with Neo4j.driver.session() as session:
            for row in df.collect():
                for node_col in node:
                    node_value = row[node_col]
                    # Lambda to create property setting part of the query
                    create_property_query = lambda props: ", ".join([f"n.{prop} = '{row[prop]}'" for prop in props]) if props else ""

                    # If properties exist, create a more detailed query
                    if properties and node_col in properties:
                        property_col = create_property_query(properties[node_col])
                        query = f"MERGE (n:{node_col} {{ {node_col}: '{node_value}' }}) SET {property_col}"
                    else:
                        query = f"MERGE (n:{node_col} {{ {node_col}: '{node_value}' }})"

                    # Run the query
                    session.run(query)

            if relationship:
                for start_node, end_node, relationship_type in relationship:
                    if start_node in df.columns and end_node in df.columns:
                        start_node_value = row[start_node]
                        end_node_value = row[end_node]

                        relationship_query = f"""
                            MATCH (a {{ {start_node}: $start_node_value }})
                            MERGE (b {{ {end_node}: $end_node_value }})
                            MERGE (a)-[:{relationship_type}]->(b)
                        """
                        session.run(relationship_query, start_node_value=start_node_value, end_node_value=end_node_value)
```

3. Lexicon Enrichment (if any)

3.1. List of enrichment carried out

- Sentiment Analysis
 - Sentiment Labelling: Assign sentiment scores to words (positive, negative, neutral).
 - Contextual Sentiment Analysis: Consider the context of a word to determine its sentiment.

3.2. List of Python classes and Jupyter notebook(s) for lexicon enrichment:

Name of Python class(es) / notebooks	Author
SentimentAnalysis.ipynb	Chan Jin Wei

3.3. Code for Python Classes

```
from pyspark.sql import SparkSession
import pandas as pd
from malaya.supervised.huggingface import load
from malaya.torch_model.huggingface import Classification
from classes.dataframe_saver import DataFrameSaver
import subprocess
```

```
subprocess.run(['hdfs', 'dfs', '-rm', 'dictionary4/*'])
```

```

spark = SparkSession.builder.appName("SentimentAnalysis").getOrCreate()

input_file = 'dictionary3/dictionary3.csv'
output_dir = '/home/student/de-assgt/content'
file_name = 'dictionary4.csv'
hdfs_path = 'dictionary4'

```

```

pyspark_df = spark.read.csv(input_file, header=True, inferSchema=True)

# Convert PySpark DataFrame to Pandas for processing
pandas_df = pyspark_df.toPandas()

# Define the Label and HuggingFace model information
label = ['negative', 'neutral', 'positive']

available_huggingface = {
    'mesolitica/sentiment-analysis-nanot5-tiny-malaysian-cased': {
        'Size (MB)': 93,
        'macro precision': 0.67768,
        'macro recall': 0.68266,
        'macro f1-score': 0.67997,
    },
    'mesolitica/sentiment-analysis-nanot5-small-malaysian-cased': {
        'Size (MB)': 167,
        'macro precision': 0.67602,
        'macro recall': 0.67120,
        'macro f1-score': 0.67339,
    }
}

# Load the HuggingFace sentiment model
model_name = 'mesolitica/sentiment-analysis-nanot5-small-malaysian-cased'
sentiment_model = load(
    model=model_name,
    class_model=Classification,
    available_huggingface=available_huggingface,
    force_check=True,
    path=None, # Use None or specify a valid path if required
)

# Perform sentiment analysis
def predict_sentiment(text):
    if pd.notnull(text): # Check if text is not NaN
        return sentiment_model.predict([text])[0]
    return None

pandas_df['sentiment'] = pandas_df['words'].apply(predict_sentiment)

result_df = spark.createDataFrame(pandas_df)

DataFrameSaver.save_to_csv(result_df, output_dir, file_name)

result_df.show()

```

4. Lexicon Analysis and Evaluation

4.1. Results of Analysis (if any)

Part-of-Speech Distribution

```
+-----+-----+-----+
| words | definition | pos_tag | freq_of_use |
+-----+-----+-----+
| perdana | [per.da.na] Defi... | - | - |
| datuk | [da.tu] Definisi... | - | - |
| anwar | - | - | - |
| ibrahim | - | - | - |
| hari | [ha.ri] Definisi... | - | - |
| ini | [i.ni] Definisi ... | - | - |
| mengadakan | [me.nga.da.kan] ... | - | - |
| pertemuan | [per.te.muan] De... | - | - |
| bersama | [ber.sa.ma] Defi... | - | - |
| kualala | [kua.la] Definis... | kata nama | High |
| lumpur | [lum.pur] Defini... | kata nama | High |
| menteri | [menteri] Defini... | kata nama | High |
| seri | [se.ri] Definisi... | kata nama | High |
| terkini | [ter.ki.ni] Defi... | adverba | Low |
| pengamal | [peng.Oa.mal][pe... | - | - |
| media | [mé.dia] Definisi... | - | - |
| tempatan | [tem.pa.tan] Def... | - | - |
| dan | - | - | - |
| bagi | [ba.gi] Definisi... | - | - |
| menyampaikan | [me.nyam.pai.kan]... | - | - |
+-----+-----+-----+
only showing top 20 rows
```

Most Common Words:

```
+-----+-----+
| words | frequency |
+-----+-----+
| dan | 9 |
| negara | 5 |
| di | 4 |
+-----+-----+
```

Least Common Words:

```
+-----+-----+
| words | frequency |
+-----+-----+
| pengamal | 1 |
| hantaran | 1 |
| terkini | 1 |
+-----+-----+
```

Part-of-Speech Distribution:

```
+-----+-----+
| pos_tag | count |
+-----+-----+
| - | 87 |
| kata nama | 48 |
| kata tugas | 14 |
| adjektif | 8 |
| kata kerja | 7 |
| adverba | 3 |
+-----+-----+
```

Most Common POS Tag (excluding '-'):

```
+-----+-----+
| pos_tag | count |
+-----+-----+
| kata nama | 48 |
+-----+-----+
```

Least Common POS Tag (excluding '-'):

```
+-----+-----+
| pos_tag | count |
+-----+-----+
| adverba | 3 |
+-----+-----+
```

Morphological Analysis

words	definition	pos_tag	freq_of_use	kata_dasar	kata_terbitan	num_variations
perdana	[per.da.na] Defini...	-	-	-	-	0
datuk	[da.tu] Definisi...	-	-	-	-	0
anwar	-	-	-	-	-	0
ibrahim	-	-	-	-	-	0
hari	[ha.ri] Definisi...	-	-	hari	sehari-hari, seha...	4
ini	[i.ni] Definisi ...	-	-	-	-	0
mengadakan	[me.nga.da.kan] ...	-	-	-	-	0
pertemuan	[per.te.muan] De...	-	-	temu	bertemu, menemui,...	7
bersama	[ber.sa.ma] Defi...	-	-	-	-	0
kuala	[kua.la] Definisi...	kata nama	High	kuala	berkuala, ke kuala	2
lumpur	[lum.pur] Defini...	kata nama	High	lumpur	-	0
menteri	[menteri] Defini...	kata nama	High	menteri	kementerian	1
seri	[se.ri] Definisi...	kata nama	High	seri	berseri, menyerikan	2
terkini	[ter.ki.ni] Defi...	adverba	Low	kini	terkini	1
pengamal	[peng.Öa.mal][pe....]	-	-	-	-	0
media	[mé.dia] Definisi...	-	-	-	-	0
tempatan	[tem.pa.tan] Def...	-	-	-	-	0
dan	-	-	-	-	-	0
bagi	[ba.gi] Definisi...	-	-	-	-	0
menyampaikan	[me.nyam.pai.kan]...	-	-	-	-	0

only showing top 20 rows

Word Length and Complexity Analysis

words	definition	pos_tag	freq_of_use	kata_dasar	kata_terbitan	num_variations	word_length	length_category
tengah-tengah	-	-	-	-	-	0	13	long
menyampaikan	[me.nyam.pai.kan] ...	-	-	-	-	0	12	long
perkembangan	[per.kem.ban.ngan] ...	kata kerja	Moderate	kembang	berkembang, menge...	5	12	long
perkongsi	[per.kong.sian] ...	kata nama	High	kongsi	berkongsi, perkon...	2	11	long
kebangkitan	[ke.bang.ki.tan] ...	-	-	-	-	0	11	long
penjelasan	[pen.je.la.san] ...	adjektif	Moderate	jelas	berjelas-jelas, m...	4	10	long
memberikan	[mem.be.ri.kan] ...	-	-	-	-	0	10	long
kemiskinan	[ke.mis.ki.nan] ...	adjektif	Moderate	misikin	memiskinkan, kemi...	2	10	long
mengadakan	[me.nga.da.kan] ...	-	-	-	-	0	10	long
pencerahan	[pen.ce.ra.han] ...	-	-	-	-	0	10	long
sewajarnya	[se.wa.jar.nya] ...	adjektif	Moderate	wajar	sewajar, sewajarnya	2	10	long
pencerahan	[pen.ce.ra.han] ...	-	-	-	-	0	10	long
memerlukan	[me.mer.lu.kan] ...	kata tugas	Moderate	perlu	memerlukan, keper...	2	10	long
berkembang	[ber.kem.bang] D...	kata kerja	Moderate	kembang	berkembang, menge...	5	10	long
berkaitan	[ber.kai.tan] De...	kata nama	High	kait	berkaitan	1	9	long
pertemuan	[per.te.muan] De...	-	-	temu	bertemu, menemui,...	7	9	long
diberikan	-	-	-	-	-	0	9	long
khususnya	-	-	-	-	-	0	9	long
perubahan	[pe.ru.ba.han] [pe...	-	-	ubah	berubah, berubah-...	5	9	long
pandangan	[pan.da.ngan] De...	-	-	pandang	berpandangan, ber...	9	9	long

only showing top 20 rows

Named Entity Analysis

[illegible]

4.4 List of Python classes and Jupyter notebook(s) for lexicon enrichment:

Name of Python class(es) / notebooks	Author
WordLengthAnalysis.ipynb	Chin Yin Ern
word_length_analysis.py	Chin Yin Ern
PartOfSpeechDist.ipynb	Kelly Tan Jie Li
part_of_speech.py	Kelly Tan Jie Li
MorphologicalAnalysis.ipynb	Chan Jin Wei
morphological_analysis.py	Chan Jin Wei
EntityAnalysis.ipynb	Kenny Loh Kit Yi
entity_processor.py	Kenny Loh Kit Yi

4.5 Code for Python Classes

word_length_analysis.py

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *

class WordLengthAnalysis:

    @staticmethod
    def performAllOperations(input_file, spark):
        # Step 1: Read the dictionary.csv file from HDFS
        df = spark.read.csv(input_file, header=True, inferSchema=True)

        # Step 2: Process word lengths
        df = df.withColumn("word_length", length(col("words"))).orderBy(col("word_length").desc())

        # Step 3: Segment words by length category
        df = df.withColumn(
            "length_category",
            when(col("word_length") <= 4, "short")
            .when(col("word_length").between(5, 8), "medium")
            .otherwise("long")
        )
        df.show()

        return df

    @staticmethod
    def calculateOverallAverageWordLength(df, spark):

        df_stats = df.withColumn("char_count_no_spaces", length(regexp_replace(col("words"), " ", ""))) \
            .withColumn("word_count", size(split(col("words"), " ")))

        total_characters = df_stats.select(sum("char_count_no_spaces").alias("total_characters")).collect()[0][0]
        total_words = df_stats.select(sum("word_count").alias("total_words")).collect()[0][0]

        overall_avg_word_length = total_characters / total_words if total_words > 0 else 0

        print(f"Total Characters (excluding spaces): {total_characters}")
        print(f"Total Words: {total_words}")
        print(f"Overall Average Word Length: {overall_avg_word_length:.2f}")

    @staticmethod
    def calculateHighestWordLength(df, spark):
        """
        Print words with the highest and lowest word lengths.
        """
        max_length = df.agg(max("word_length")).collect()[0][0]
        min_length = df.agg(min("word_length")).collect()[0][0]

        max_length_df = df.filter(col("word_length") == max_length)
        min_length_df = df.filter(col("word_length") == min_length)

        print("Words with maximum length:")
        max_length_df.select("words", "word_length").show()

        print("Words with minimum length:")
        min_length_df.select("words", "word_length").show()
```

part_of_speech.py

```

import os
import requests
from bs4 import BeautifulSoup
import csv
import subprocess
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, count, when, lit, desc
from classes.dataframe_saver import DataFrameSaver

class FindPartOfSpeech:

    @staticmethod
    def get_content_from_page(base_url, word):
        url = f"{base_url}{word}"
        response = requests.get(url)

        if response.status_code != 200:
            print(f"Failed to fetch {url}, Status Code: {response.status_code}")
            return "-"

        soup = BeautifulSoup(response.content, 'html.parser')
        pos_tags = [] # List to store POS tags

        try:
            # Check if the word has no thesaurus information
            no_info_tag = soup.find('b', string=lambda text: text and "Tiada maklumat tesaurus" in text)
            if no_info_tag:
                return "-"

            # Use a set to store unique POS tags
            types = set()

            # Extract type information from <font> under <i>
            for font in soup.find_all('font', color='blue'):
                italic = font.find('i')
                if italic:
                    types.add(italic.get_text(strip=True).rstrip(',').rstrip(':'))

            # Extract type information from <i> under <font>
            for italic in soup.find_all('i'):
                font = italic.find('font', color='blue')
                if font:
                    types.add(font.get_text(strip=True).rstrip(',').rstrip(':'))

            # Return sorted List of POS tags
            pos_tags = sorted(types)

        except Exception as e:
            print(f"Error parsing word '{word}': {e}")

        return pos_tags

```

```

@staticmethod
def process_pos(input_file, base_url, output_dir, file_name, spark):
    os.makedirs(os.path.dirname(input_file), exist_ok=True)

    # Read the input CSV file from HDFS
    df = spark.read.csv(input_file, header=True, inferSchema=True)

    # Extract rows as a list of dictionaries for processing POS tags
    rows = df.collect()
    header = ['words', 'definition', 'pos_tag']
    data = []

    for row in rows:
        words = row['words']
        definition = row['definition'] if row['definition'] else "-"
        pos_tags = FindPartOfSpeech.get_content_from_page(base_url, words)

        if pos_tags:
            for pos in pos_tags:
                data.append([words, definition, pos])
        else:
            data.append([words, definition, "-"])

    # Convert processed data into a DataFrame
    df = spark.createDataFrame(data, schema=header)

    # Compute freq_of_use
    pos_counts = df.groupBy("pos_tag").agg(count("*").alias("count"))
    filtered_pos_counts = pos_counts.filter(col("pos_tag") != "-")

    max_count = filtered_pos_counts.agg({"count": "max"}).collect()[0][0]
    min_count = filtered_pos_counts.agg({"count": "min"}).collect()[0][0]

    pos_counts_with_freq = pos_counts.withColumn(
        "freq_of_use",
        when(col("pos_tag") == "-", lit("-")) # Retain '-' as '-'
        .when(col("count") == max_count, lit("High"))
        .when(col("count") == min_count, lit("Low"))
        .otherwise(lit("Moderate"))
    )

    df_with_freq = df.join(
        pos_counts_with_freq.select("pos_tag", "freq_of_use"),
        on="pos_tag",
        how="left"
    )

    # Save the updated DataFrame back to a Local CSV file
    final_columns = header + ["freq_of_use"]
    df_with_freq_ordered = df_with_freq.select(*final_columns)
    local_output_file = os.path.join(output_dir, file_name)
    DataFrameSaver.save_to_csv(df_with_freq_ordered, output_dir, file_name)

    return df_with_freq_ordered

```

morphological_analysis.py

```
from pyspark.sql.functions import *
import re
import requests
from bs4 import BeautifulSoup
import matplotlib.pyplot as plt
from pyspark.sql.types import StringType
from classes.dataframe_saver import DataFrameSaver

class MorphologicalAnalysis:

    @staticmethod
    def fetch_kata_terbitan(word):
        url = f"https://prpm.dbp.gov.my/Cari1?keyword={word}"
        response = requests.get(url)

        if response.status_code == 200:
            soup = BeautifulSoup(response.content, 'html.parser')

            # Locate the table with 'Kata Terbitan'
            kata_terbitan_section = soup.find('b', string="Kata Terbitan : ")

            if kata_terbitan_section:
                # Find all <a> tags under the <i> tag that follows <b>Kata Terbitan :</b>
                kata_terbitan_links = kata_terbitan_section.find_next('i').find_all('a')

                # Extract text from each link
                kata_terbitan_list = [link.get_text(strip=True) for link in kata_terbitan_links]

                # Return comma-separated list of 'kata terbitan' or a fallback message
                return ', '.join(kata_terbitan_list) if kata_terbitan_list else "No kata terbitan found"
            else:
                return "-"
        else:
            return f"Failed to fetch URL, Status Code: {response.status_code}"

    @staticmethod
    def clean_numbers(text):
        return re.sub(r'\d+', '', text).strip()

    @staticmethod
    def fetch_kata_dasar(word):
        url = f"https://prpm.dbp.gov.my/Cari1?keyword={word}"
        response = requests.get(url)

        if response.status_code == 200:
            soup = BeautifulSoup(response.content, 'html.parser')

            # Locate all <span> elements with 'font-weight:bold;'
            kata_dasar = soup.find_all('span', style=lambda value: value and 'font-weight:bold;' in value)

            # If span is found, return its cleaned text directly
            if len(kata_dasar) == 1:
                return MorphologicalAnalysis.clean_numbers(kata_dasar[0].get_text(strip=True))

            if not kata_dasar: # No matching spans
                return "-"
        else:
            return f"Failed to fetch URL, Status Code: {response.status_code}"
```

```

@staticmethod
def process_dataframe(df):
    """
    Process the DataFrame to add kata_dasar, kata_terbitan, and num_variations columns.
    """
    # Define UDFs to process the words
    kata_dasar_udf = udf(lambda word: MorphologicalAnalysis.fetch_kata_dasar(word), StringType())
    kata_terbitan_udf = udf(lambda word: MorphologicalAnalysis.fetch_kata_terbitan(word), StringType())

    # Add kata_dasar and kata_terbitan columns
    df = df.withColumn('kata_dasar', kata_dasar_udf(col('words')))
    df = df.withColumn('kata_terbitan', kata_terbitan_udf(col('words')))

    # Add num_variations column
    df = df.withColumn(
        'num_variations',
        when((col('kata_terbitan').isNotNull()) & (col('kata_terbitan') != '-'), size(split(col('kata_terbitan'), ',')))
        .otherwise(0)
    )

    return df

@staticmethod
def summary(df):
    """
    Generates a summary of the processed DataFrame.
    Includes:
    - Counts of words with/without 'kata_terbitan'
    - Distribution of 'num_variations'
    - Words with the maximum number of derivations
    - Visualization of derivation distribution
    """
    # Clean and trim whitespace for 'kata_terbitan' column
    df = df.withColumn("kata_terbitan", trim(col("kata_terbitan")))

    # Count words with null and non-null 'kata_terbitan'
    null_count = df.filter(col("kata_terbitan") == "-").count()
    non_null_count = df.filter(col("kata_terbitan") != "-").count()

    # Add a column for the number of derivations (split by ',' if valid)
    derivation_distribution = (
        df.groupBy("num_variations")
        .agg(count("*").alias("frequency"))
        .orderBy("num_variations")
    )

    # Visualization using Matplotlib
    distribution_data = derivation_distribution.collect()
    x_values = [row["num_variations"] for row in distribution_data]
    y_values = [row["frequency"] for row in distribution_data]

    plt.figure(figsize=(10, 6))
    plt.bar(x_values, y_values, color='skyblue')
    plt.title("Distribution of 'kata_terbitan' Derivations")
    plt.xlabel("Number of Derivations")
    plt.ylabel("Frequency")
    plt.xticks(x_values)
    plt.grid(axis='y', linestyle='--', alpha=0.7)
    plt.tight_layout()
    plt.show()

    # Determine the maximum number of derivations
    max_derivations = df.agg(max("num_variations")).collect()[0][0]

    # Filter rows where 'num_variations' equals the maximum value
    max_derivations_df = df.filter(col("num_variations") == max_derivations)

    # Print summary statistics
    print(f"Number of words with no 'kata_terbitan' : {null_count}")
    print(f"Number of words with 'kata_terbitan' : {non_null_count}\n")

    if max_derivations > 0:
        print(f"Words with the highest number of 'kata_terbitan' derivations ({max_derivations}):")
        max_derivations_df.select("words", "kata_terbitan", "num_variations").show(truncate=False)
    else:
        print("No valid derivations found.")

    print("Original data frame:")
    df.show()

```

entity_processor.py

```
import requests

class EntityProcessor:
    @staticmethod
    def get_entity_summaries(titles):
        url = "https://ms.wikipedia.org/w/api.php"
        params = {
            "action": "query",
            "format": "json",
            "titles": "|".join(titles),
            "prop": "categories|extracts",
            "cllimit": "max",
            "exintro": True,
            "explaintext": True
        }
        response = requests.get(url, params=params)
        data = response.json()

        results = {}
        for page_id, page in data["query"]["pages"].items():
            if "missing" in page:
                continue

            # Extract categories
            categories = [cat["title"] for cat in page.get("categories", [])]

            # Define category keywords
            location_keywords = ["bandar", "kampung", "negeri", "wilayah", "daerah", "kampung", "jalan"]
            people_keywords = ["orang", "individu", "lahir", "biografi", "kelahiran"]
            organization_keywords = ["syarikat", "perusahaan", "organisasi", "korporasi"]
            product_keywords = ["produk", "jenama", "barang"]

            # Determine entity type
            is_location = any(keyword in cat.lower() for cat in categories for keyword in location_keywords)
            is_people = any(keyword in cat.lower() for cat in categories for keyword in people_keywords)
            is_organization = any(keyword in cat.lower() for cat in categories for keyword in organization_keywords)
            is_product = any(keyword in cat.lower() for cat in categories for keyword in product_keywords)

            # Extract summary
            summary = page.get("extract", "")
            first_paragraph = summary.split("\n")[0] if summary else ""

            # Assign entity type using match
            title = page["title"]
            match (is_people, is_location, is_organization, is_product):
                case (True, _, _, _):
                    results[title] = ("People", first_paragraph, categories)
                case (_, True, _, _):
                    results[title] = ("Location", first_paragraph, categories)
                case (_, _, True, _):
                    results[title] = ("Organisation", first_paragraph, categories)
                case (_, _, _, True):
                    results[title] = ("Product", first_paragraph, categories)
                case _:
                    results[title] = (None, None, None)

        return results

    @staticmethod
    def normalize_word(word):
        return word[0].upper() + word[1:].lower()

    @staticmethod
    def generate_phrases(words, max_length):
        phrases = []
        for i in range(len(words)):
            for length in range(2, max_length + 1):
                if i + length <= len(words):
                    phrase = "_".join([EntityProcessor.normalize_word(words[j]) for j in range(i, i + length)])
                    phrases.append(phrase)
        return phrases
```

6. Data Streaming

6.1 Kafka Streaming

Use case:	Stream from raw data to HDFS
Kafka Topic:	article
End user(s):	Data scientist
	Data engineer
	Machine Learning Engineers

Screenshot of message content:

```
student@LAPTOP-PFPL3CLD X +
student@LAPTOP-PFPL3CLD:~/de-assgt$ cd ~
student@LAPTOP-PFPL3CLD:~$ source de-assgt/de-assgt/bin/activate
(de-assgt) student@LAPTOP-PFPL3CLD:~$ cd de-assgt/classes
(de-assgt) student@LAPTOP-PFPL3CLD:~/de-assgt/classes$ python producer.py
24/12/22 18:13:45 WARN Utils: Your hostname, LAPTOP-PFPL3CLD, resolves to a loopback address: 127.0.1.1; using
g 10.255.255.254 instead (on interface lo)
24/12/22 18:13:45 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
24/12/22 18:13:46 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using built
in java classes where applicable
Processing category: https://www.sinarharian.com.my/nasional
Article content saved as articles.csv (local)
Article content has been stored in hdfs
Article 1:
KUALA LUMPUR - Jabatan Meteorologi Malaysia (MetMalaysia) mengeluarkan amaran hujan berterusan tahap waspada
bagi beberapa kawasan di Sabah dan Labuan hari ini. Ketua Pengarah MetMalaysia, Dr Mohd Hisham Mohd Anip dala
m kenyataan berkata, di Sabah, ia melibatkan kawasan Pedalaman iaitu Kuala Penyu dan Beaufort. Beliau berkata
, bagi Pantai Barat, kawasan yang terlibat adalah Papar, Putatan, Penampang, Kota Kinabalu, Tuaran dan Kota B
elud selain Kudat. Amaran sama turut melibatkan Labuan. Semalam, MetMalaysia mengeluarkan amaran hujan berter
usan tahap buruk bagi kawasan Kuala Penyu dan Beaufort. "Orang ramai boleh mendapatkan maklumat cuaca terkini
melalui laman web rasmi Jabatan, aplikasi mobil myCuaca dan laman...

Processing category: https://www.sinarharian.com.my/nasional
Article content saved as articles.csv (local)
Article content has been stored in hdfs
Article 2:
BANTING - Polis Diraja Malaysia (PDRM) menahan enam lelaki warga tempatan dipercayai terlibat dalam kejadian
rusuhan di Jenjarom, di sini pada Khamis. Ketua Polis Daerah Kuala Langat, Superintenden Mohd Akmalrizal Radz
i berkata, pihaknya menerima panggilan daripada seorang lelaki warga tempatan yang merupakan seorang doktor d
i Hospital Banting pada jam 10.50 malam, Khamis. Beliau berkata, doktor tersebut melaporkan menerima seorang
pesakit lelaki warga tempatan mengalami kecederaan serius di bahagian kepala dan berada di Zon Merah, Hospita
l Banting. "Sepasukan polis daripada Bahagian Siasatan Jenayah Ibu Pejabat Polis Daerah (IPD) Kuala Langat be
rsama Jabatan Siasatan Jenayah Ibu Pejabat Polis Kontinjen (IPK) Selangor telah...

Processing category: https://www.sinarharian.com.my/nasional
Article content saved as articles.csv (local)
Article content has been stored in hdfs
Article 3:
SHAH ALAM - Sinar Harian bakal membuka tirai tahun 2025 dengan program jelajah berkonsepkan tanggungjawab sos
ial korporat (CSR) bagi membantu dan memperkasakan kelompok masyarakat di seluruh negara iaitu Jelajah Sinar
Untuk Rakyat. Ketua Pengarang Kumpulan Sinar Harian, Zamri Rambli berkata, lokasi pertama jelajah berkenaan a
kan diadakan di Masjid Jamek Baseri dan Institut Kemahiran Mara (IKM) Baseri, Perlis pada 4 dan 5 Januari dep
```



```

student@LAPTOP-PFPL3CLD X +
student@LAPTOP-PFPL3CLD:~/de-assgt$ cd ~
student@LAPTOP-PFPL3CLD:~$ source de-assgt/de-assgt/bin/activate
(de-assgt) student@LAPTOP-PFPL3CLD:~$ cd de-assgt/classes
(de-assgt) student@LAPTOP-PFPL3CLD:~/de-assgt/classes$ python consumer.py
Article 1:
KUALA LUMPUR - Jabatan Meteorologi Malaysia (MetMalaysia) mengeluarkan amaran hujan berterusan tahap waspada bagi beberapa kawasan di Sabah dan Labuan hari ini. Ketua Pengarah MetMalaysia, Dr Mohd Hisham Mohd Anip dalam kenyataan berkata, di Sabah, ia melibatkan kawasan Pedalaman iaitu Kuala Penyu dan Beaufort. Beliau berkata, bagi Pantai Barat, kawasan yang terlibat adalah Papar, Putatan, Penampang, Kota Kinabalu, Tuaran dan Kota Belud selain Kudat. Amaran sama turut melibatkan Labuan. Semalam, MetMalaysia mengeluarkan amaran hujan berterusan tahap buruk bagi kawasan Kuala Penyu dan Beaufort. "Orang ramai boleh mendapatkan maklumat cuaca terkini melalui laman web rasmi Jabatan, aplikasi mobil myCuaca dan laman..."

Article 2:
BANTING - Polis Diraja Malaysia (PDRM) menahan enam lelaki warga tempatan dipercayai terlibat dalam kejadian rusuhan di Jenjarom, di sini pada Khamis. Ketua Polis Daerah Kuala Langat, Superintenden Mohd Akmalrizal Radzi berkata, pihaknya menerima panggilan daripada seorang lelaki warga tempatan yang merupakan seorang doktor di Hospital Banting pada jam 10.50 malam, Khamis. Beliau berkata, doktor tersebut melaporkan menerima seorang pesakit lelaki warga tempatan mengalami kecederaan serius di bahagian kepala dan berada di Zon Merah, Hospital Banting. "Sepasukan polis daripada Bahagian Siasatan Jenayah Ibu Pejabat Polis Daerah (IPD) Kuala Langat bersama Jabatan Siasatan Jenayah Ibu Pejabat Polis Kontinjen (IPK) Selangor telah..."

Article 3:
SHAH ALAM - Sinar Harian bakal membuka tirai tahun 2025 dengan program jelajah berkonsepkan tanggungjawab sosial korporat (CSR) bagi membantu dan memperkasakan kelompok masyarakat di seluruh negara iaitu Jelajah Sinar Untuk Rakyat. Ketua Pengarang Kumpulan Sinar Harian, Zamri Rambli berkata, lokasi pertama jelajah berkenaan akan diadakan di Masjid Jamek Besei dan Institut Kemahiran Mara (IKM) Besei, Perlis pada 4 dan 5 Januari depan. "Jelajah di negeri paling utara Semenanjung itu bakal disempurnakan majlis perasmian oleh Raja Muda Perlis, Tuanku Syed Faizuddin Putra Jamalullail. "Program kenduri sekampung, penyerahan bantuan kepada asnaf dan ceramah motivasi kepada pelajar itu diadakan dengan kolaborasi penaja..."

Article 4:
KUALA LUMPUR - Perdana Menteri, Datuk Seri Anwar Ibrahim merakamkan ucapan takziah kepada keluarga aktivis gerakan Islam Mesir Youssef Nada yang meninggal dunia hari ini. "Sedih tatkala sahabat keluarga, aktivis gerakan Islam berasal Mesir serta juga pengusaha terkenal Youssef Nada telah kembali menyahut seruan Ilahi. "Bagi pihak keluarga, kami berduka dan mendoakan agar rohnya diberi keampunan serta dikurniakan tempat istimewa di sisi-Nya, insya-Allah," kata beliau menerusi hantaran di Facebook pada Ahad. Nada, 93, merupakan ahli perniagaan dan bapa kewangan Islam serta pengasas Al Taqwa Islamic Bank. Beliau terkenal kerana kejayaan kes perundangan hak asasi manusia di Eropah untuk mempertahankan dirinya..."

^CTraceback (most recent call last):
  File "/home/student/de-assgt/classes/consumer.py", line 15, in <module>
    for msg in consumer:

```

List of Python classes for Kafka producer and consumer:

Name of Python class(es) / notebooks	Author
producer.py	Kelly Tan Jie Li
consumer.py	Kelly Tan Jie Li

6.2 Python Code

producer.py

```
from kafka import KafkaProducer
import time
from Stream_Process import stream_process

bootstrap_servers = "localhost:9092"
topic = 'article'
time_interval = 2

producer = KafkaProducer(bootstrap_servers=bootstrap_servers)

for num in range(1, 1000): # Adjust range for desired article count
    message = stream_process.api_crawl(num).encode('utf-8')
    if message.decode('utf-8') == "---":
        break # Stop if no more valid articles
    print(message.decode('utf-8')) # Print the sent message for reference
    producer.send(topic, message)
    time.sleep(time_interval)

producer.flush()
```

consumer.py

```
from kafka import KafkaConsumer

bootstrap_servers = "localhost:9092"
topic = 'article'

consumer = KafkaConsumer(topic, bootstrap_servers=bootstrap_servers)

for msg in consumer:
    message = msg.value.decode('utf-8')
    print(message)
```

6.3 Spark Structured Streaming

Use case:	Streaming data from csv file into neo4j
Source:	Csv file
Sink:	neo4j

End user(s):	Linguists
--------------	-----------

Screenshot of aggregated data that will be streamed:

words	definition	pos_tag	freq_of_use	kata_dasar	kata_terbitan	num_variations	word_length	length_category	sentiment
ulasan	amuz Pelajar Edisi Kedua	kata nama	High	ulas	an, pengulasan, pengulas	10	6	medium	neutral
polisi	amuz Pelajar Edisi Kedua	-	-	-	-	0	6	medium	neutral
perubahan	amuz Pelajar Edisi Kedua	-	-	ubah	perubahan, perubahan	5	9	long	neutral
kritikan	amuz Pelajar Edisi Kedua	-	-	kritikan	an, mengkritik, pengkritik	3	8	medium	negative
sewajarnya	amuz Pelajar Edisi Kedua	adjektif	Moderate	vajar	sewajar, sewajarnya	2	10	long	negative
maju	amuz Pelajar Edisi Kedua	kata kerja	Moderate	maju	jukan, kemajuan, pemaju	3	4	short	neutral
dipayakan	-	-	-	-	-	0	9	long	negative
mengena	amuz Pelajar Edisi Kedua	kata kerja	Moderate	kena	ngena-enganakan, terkena	6	8	medium	neutral
serantau	isi Dewan Edisi Keempat	-	-	-	-	0	8	medium	neutral
pandangan	amuz Pelajar Edisi Kedua	-	-	pandang	andangan, pemandangan	9	9	long	neutral
hantaran	amuz Pelajar Edisi Kedua	-	-	hantar	enghantaran, penghantar	6	8	medium	neutral
menurut	amuz Pelajar Edisi Kedua	-	-	turut	funutkan, funutan, perurut	8	7	medium	neutral
manfaat	amuz Pelajar Edisi Kedua	kata nama	High	manfaat	imantfaat, memanfaatkan	2	7	medium	positive
lagi	amuz Pelajar Edisi Kedua	kata tugas	Moderate	lagi	selagi, lagi-lagi, lagikan	3	4	short	neutral
diberikan	-	-	-	-	-	0	9	long	positive
penjelasan	amuz Pelajar Edisi Kedua	adjektif	Moderate	jelas	an, kejelasan, penjelasan	4	10	long	neutral
lebih	amuz Pelajar Edisi Kedua	kata tugas	Moderate	lebih	hi, melebihi, kelebihan	9	5	medium	positive
dan	-	-	-	-	-	0	3	short	neutral
memberikan	amuz Pelajar Edisi Kedua	-	-	-	-	0	10	long	neutral
berkembang	amuz Pelajar Edisi Kedua	kata kerja	Moderate	kembang	mbangan, pengembangan	5	10	long	neutral
kuala	amuz Pelajar Edisi Kedua	kata nama	High	kuala	berkuala, ke kuala	2	5	medium	neutral
langkah-langkah	-	-	-	-	-	0	13	long	neutral
industri	amuz Pelajar Edisi Kedua	kata nama	High	industri	-	0	8	medium	neutral
menteri	amuz Pelajar Edisi Kedua	kata nama	High	menteri	kementerian	1	7	medium	neutral
sesi	isi Dewan Edisi Keempat	kata nama	High	sesi	-	0	4	short	neutral
tan-tan	-	-	-	-	-	0	9	long	neutral
kepada	amuz Pelajar Edisi Kedua	kata tugas	Moderate	kepada	-	0	6	medium	neutral
menyampaikan	amuz Pelajar Edisi Kedua	-	-	-	-	0	12	long	neutral
teguran	amuz Pelajar Edisi Kedua	-	-	tegur	tegur, teguran, keteguran	4	7	medium	neutral

List of Python classes / Jupyter notebook(s) for Structured Streaming:

Name of Python class(es) / notebooks	Author
Neo4j.ipynb	Chin Yin Ern

6.4 Python Code

neo4j.ipynb

```
df = spark.readStream.option("header", "true").option("delimiter", "\t").schema(schema).csv(file_path)
```

```
# Start the query
query = df.writeStream \
    .outputMode("update") \
    .foreachBatch(lambda df, epoch_id: Neo4j.write_to_neo4j(df, epoch_id, node, relationship, properties)) \
    .start()
```